C++ Programming Textbook Notes

Module: Classes and Objects | Encapsulation, Scope Resolution & Memory Layout

Author: Gaurav Kandpal **Platform:** CS-Simplified **Level:** Beginner to Advanced

1. CLASSES AND OBJECTS: FOUNDATIONS & CORE CONCEPTS

What is a Class?

A **Class** is a user-defined blueprint or template that binds data members (variables) and member functions (behaviors) into a single encapsulated structure. It represents an abstract custom data type.

What is an Object?

An **Object** is a concrete runtime instance of a class. In procedural languages like C, instances of types are simply called variables. In C++, when a variable is instantiated from a `class`, it is called an **Object** because it represents an independent entity with state (data) and behavior (functions).

The Fundamental Difference Between `class` and `struct`

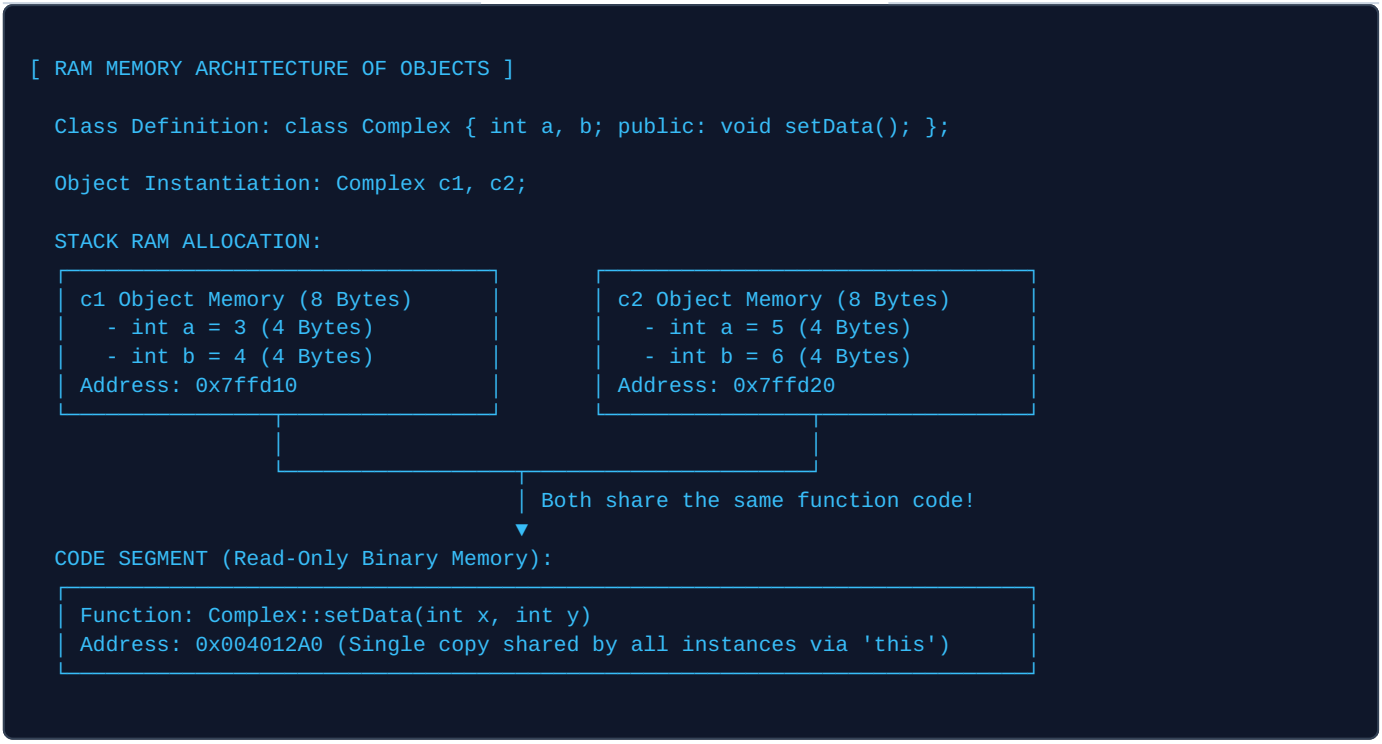
In C++, a class and a structure have identical technical capabilities—both support member variables, methods, constructors, and inheritance. **The only fundamental difference** is default access specifiers:

- In a **`struct`**, all members are **`public` by default**.
- In a **`class`**, all members are **`private` by default** (enforcing data security).

How Classes and Objects Work in Memory

Understanding how classes and objects exist in RAM is foundational for mastering C++:

- **Class Definition (0 Bytes in RAM):** Defining a class allocates zero bytes of data memory at runtime. It serves purely as a compiler type schema.
- **Object Instantiation (RAM Allocation):** When an object is instantiated (e.g., `Complex c1;`), memory is allocated on the Stack or Heap to store only its *data members*.
- **Member Functions Memory Sharing:** Member functions are **not duplicated** for each object! The machine code of member functions is loaded once into the read-only Code/Text Segment. Every object shares the exact same function code in memory.



2. SYNTAX & DEFINITION RULES

Basic Class Syntax

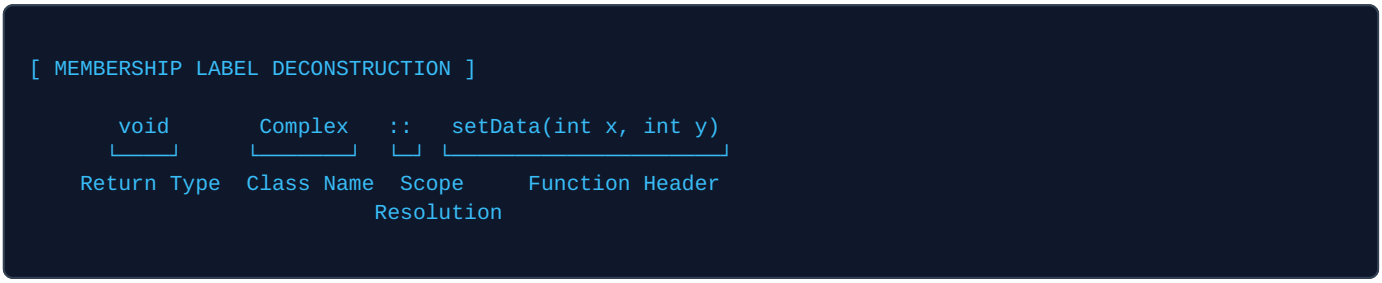
```cpp
class ClassName {
private:
    // Data members (accessible only within class member functions)
    int a, b;

public:
    // Member functions (accessible from outside the class interface)
    void setData(int x, int y);
    void showData();
};
```

Defining Member Functions Outside the Class (Membership Label)

Member functions can either be defined inside the class body or declared inside and defined outside. When defining outside:

1. The function must first be **declared inside** the class body.
2. Outside the class, define the function using the **Membership Label** with the **Scope Resolution Operator (`::`)**:
   `return_type ClassName::functionName(arguments) { /* body */ }`



> **Important Compiler Rule: In-Line Member Functions**
> Any member function whose body is defined directly **inside the class body** is treated by the compiler as **implicitly inline**! Defining large functions outside the class using `::` avoids unnecessary code bloat.

3. COMPLETE PROGRAM: BASIC CLASS & OBJECT IMPLEMENTATION

```cpp
// Basic Class and Object Demonstration in C++
#include <iostream>
using namespace std;

class Complex {
private:
    int a, b; // Private instance variables (Encapsulated)

public:
    // Member function declared and defined inside class
    void setData(int x, int y) {
        a = x;
        b = y;
    }

    // Member function declaration (Defined outside)
    void showData();
};

// Defining member function outside the class using Scope Resolution Operator (::)
void Complex::showData() {
    cout << "a = " << a << " | b = " << b << endl;
}

int main() {
    // Instantiating object 'c1' of user-defined type 'Complex'
    Complex c1;

    // Calling member functions via dot (.) operator
    c1.setData(3, 4);
    c1.showData();

    return 0;
}
```

4. PASSING AND RETURNING OBJECTS: CALLER VS. ARGUMENT OBJECTS

The Problem: Arithmetic on User-Defined Types

In C/C++, standard arithmetic operators (such as `+` or `-`) work natively only on primitive built-in types (e.g., `int`, `float`). You cannot directly write `c3 = c1 + c2;` without defining custom Operator Overloading. Attempting to do so triggers a compile-time error.

The Solution: Member Functions Taking & Returning Objects

To combine user-defined objects, we define a member function that accepts another object as an argument and returns a newly constructed object: `Complex add(Complex c);`.

Understanding Caller Objects vs. Argument Objects in Memory

When executing `c3 = c1.add(c2);`:

- **`c1` is the Caller Object:** The member function `add()` is invoked in the context of `c1`. Inside `add()`, writing variable names a and b directly accesses `c1.a` and `c1.b` through the implicit `this` pointer.
- **`c2` is the Argument Object:** Passed as a parameter. Its data members must be accessed explicitly using the dot operator: `c.a` and `c.b`.
- **`temp` is the Local Return Object:** Holds the resulting values (`temp.a = a + c.a;`) and is returned by value to assign into `c3`.

```
[ CALLER OBJECT VS ARGUMENT OBJECT INTERACTION ]

  Statement: c3 = c1.add(c2);

  Caller Object: c1 ──► Executes add() ──► Direct access to 'a' and 'b' (c1's data)
  Passed Object: c2 ──► Parameter 'c'   ──► Dot operator access: 'c.a' and 'c.b'

  Calculation inside add():
    temp.a = a + c.a;  // (c1.a + c2.a) -> (3 + 5) = 8
    temp.b = b + c.b;  // (c1.b + c2.b) -> (4 + 6) = 10
  Return temp ──► Copies result into c3
```

5. PRACTICAL CODE DEMONSTRATION: ADDING COMPLEX NUMBERS

```cpp
// Complete Complex Number Addition Program
#include <iostream>
using namespace std;

class Complex {
private:
    int a, b;

public:
    void setData(int x, int y) {
        a = x;
        b = y;
    }

    void showData() {
        cout << "Real: " << a << " | Imaginary: " << b << endl;
    }

    // Member function taking an object argument and returning an object
    Complex add(Complex c) {
        Complex temp;
        // 'a' and 'b' refer directly to caller object (c1)
        // 'c.a' and 'c.b' refer to argument object (c2)
        temp.a = a + c.a;
        temp.b = b + c.b;
        return temp;
    }
};

int main() {
    Complex c1, c2, c3;

    c1.setData(3, 4);
    c2.setData(5, 6);

    // c1 is caller object; c2 is passed argument; result assigned to c3
    c3 = c1.add(c2);

    cout << "Result of addition in c3:" << endl;
    c3.showData(); // Outputs: Real: 8 | Imaginary: 10

    return 0;
}
```

6. SUMMARY COMPARISON & BEST PRACTICES

Feature	Caller Object (c1)	Argument Object (c2)
Invocation Role	Calls the member function (`c1.add(...)`).	Passed inside argument parentheses.
Member Access Syntax	Direct access without dot operator (a, b).	Requires dot operator on parameter name (c.a, c.b).
Internal Mechanism	Bound through the implicit `this` pointer.	Passed as a standard parameter on the stack frame.

Common Gotcha: Referencing Caller Objects by Name

Inside `Complex::add(Complex c)`, beginners often attempt to write `temp.a = c1.a + c2.a;`. This fails because identifiers `c1` and `c2` are local to `main()` and completely out of scope inside the class method!

Best Practice: Pass Objects by Const Reference

In production C++, pass object parameters by **Const Reference** (e.g., `Complex add(const Complex &c) const`) to prevent creating redundant stack copies while maintaining read-only safety.